ShopDesk

Inventory and point-of-sale for small shops

Project documentation

Part 1. Mobile app (Flutter)

Part 2. Backend server (Node.js and Neon Postgres)

Part 3. Deployment guide

Flutter · Riverpod · SQLite · Node.js · Express · Neon Postgres · Vercel

ShopDesk: mobile app (Flutter)

Inventory and point-of-sale for small shops. Sell, track stock and see what to reorder. Mobile first, adapts to tablets and desktop. Shop data works offline.

The matching backend (sign-up, email codes, password reset) is in `../shopdesk_server`.

What it does

Area	Features
Account	Owner sign-up with an emailed 6-digit code, sign in, forgot password (code first, then new password)
Home	Sales today, orders today, stock value, items needing restock, weekly sales chart, running-low list
Sell	Products by category, search, cart, discount %, tax, payment type, customer name, PDF receipt
Stock	Add, edit, delete, restock; **scan stock in** with a USB scanner or the phone camera
Products	Icon from the name, emoji picker, photo from camera or gallery, optional online photo lookup
Sales	History by day / 7 / 30 days, items sold, average order, reopen any receipt
Add sale	Enter a sale by hand: scan or pick items, tax, payment, custom date, receipt
Predictor	Best and worst sellers, days of stock left, reorder advice with suggested quantity
Settings	Shop name, currency, default tax rate, receipt footer
Receipts	80 mm PDF, print or share

The shop starts empty. No sample products or sales are added.

Tech stack

- **Flutter 3** (Dart 3.4+), Material 3, light and dark themes
- **State:** Riverpod (`Notifier`, `AsyncNotifier`)
- **Navigation:** go_router with an auth redirect
- **Local data:** SQLite (`sqflite`), settings in `shared_preferences`
- **Sign-in session:** `flutter_secure_storage` (Keychain / Android Keystore)
- **Scanning, images, receipts:** `mobile_scanner`, `image_picker`, `pdf` + `printing`
- **Network:** `http`, talking to the ShopDesk Node.js server

Run it

```
flutter create . --org com.yourcompany --project-name shopdesk   # generates android/ ios/
(keeps lib/)
flutter pub get
flutter run
flutter test
```

`flutter create .` only adds the platform folders. It keeps your `lib/`.

Connect to your backend

The app talks to your server through one build setting, `API_URL`:

```
flutter run --dart-define=API_URL=https://your-server.vercel.app
flutter build apk --release --dart-define=API_URL=https://your-server.vercel.app
```

Or set a default once, in `lib/core/config/app_config.dart`:

```
static const apiUrl = String.fromEnvironment(
  'API_URL',
  defaultValue: 'https://your-server.vercel.app',
);
```

Local testing on the Android emulator: `--dart-define=API_URL=http://10.0.2.2:3000` (plain `http` needs `android:usesCleartextTraffic="true"` in the debug manifest only).

Mode	When	Emails
Your server	`API_URL` is set	Real emails sent by `shopdesk_server`
Supabase	Supabase keys given, no `API_URL`	Real emails through Supabase
Demo	nothing set	None. The code is shown on screen so you can test the screens

Details: `docs/AUTH_SETUP.md`.

Platform setup (after `flutter create .`)

Android: `android/app/src/main/AndroidManifest.xml`, inside `<manifest>`:

```
<uses-permission android:name="android.permission.CAMERA"/>
<uses-permission android:name="android.permission.INTERNET"/>
```

Check the `mobile_scanner` README for the minimum `minSdkVersion`.

iOS: `ios/Runner/Info.plist`:

```
<key>NSCameraUsageDescription</key><string>Scan product barcodes and take product photos.
</string>
<key>NSPhotoLibraryUsageDescription</key><string>Choose product photos.</string>
```

How sign-in works in the app

1. **Create owner account:** email + password. The server emails a 6-digit code.
2. **Verify:** typing the code signs the owner in. Sign-in is refused until the email is verified.
3. **Session:** a refresh token is kept in secure storage. If the phone is offline when the app opens, the saved owner stays signed in so the shop can keep selling. A clear "session ended" answer from the server signs the owner out.
4. **Forgot password:** email, then code, then the new-password screen. That screen cannot be reached without the correct code. Changing the password signs out every other phone.
5. **Create account** is always shown when using your server. The server decides, and answers "Creating new accounts is turned off for this shop" once the owner exists.

Architecture

Feature-first, with layers inside each feature. Pages never touch the database or the network directly.

```
lib/
  core/          theme, router, database, config, shared widgets, formatters
  features/
    auth/        domain · data (HTTP server, Supabase, demo) · application · presentation
    products/    domain · data (SQLite, image lookup) · application · presentation
    sales/       domain (Sale, PricingService) · data (SQLite) · application · presentation
    scanner/     scan input bar (hardware scanner or camera) and scanner page
    dashboard/   weekly sales provider and page
    predictor/   ForecastService (pure Dart, unit tested) and page
    settings/    SettingsController (SharedPreferences) and page
    shell/       adaptive navigation (bottom bar on phones, side rail on tablets)
```

- **Swappable backends.** `AuthRepository`, `ProductRepository` and `SaleRepository` are interfaces. Auth has three implementations (`HttpAuthRepository`, `SupabaseAuthRepository`, `LocalDemoAuthRepository`), chosen in `auth_controller.dart`. No screen changes when you switch.
- **One sale model.** `SaleDraft` serves both Sell and Add sale, and `PricingService` does all the maths.
- **Error handling.** Every backend maps its errors to one `AuthFailure` type with a message safe to show.

Tests

```
flutter test
```

- `pricing_service_test.dart`: totals, discount and tax maths
- `forecast_service_test.dart`: sell-through, days left, reorder advice
- `auth_flow_test.dart`: demo sign-up, codes, expiry, retries, reset rules
- `http_auth_repository_test.dart`: server calls, error mapping, offline restore, reset token rules

Before you ship

- Build with `API_URL` pointing to your **https** server.
- Install fresh on test phones (uninstall old test builds so no old sample data remains).
- Money is stored as `double` and rounded to 2 places. For strict accounting, move to integer minor units.
- The online photo lookup uses Open Food Facts (mostly food). Do the lookup on your server and cache the image.
- The PDF receipt uses the currency code because the built-in PDF font has no currency symbols. Embed a font with `pdf` for symbols.
- Shop data (products, sales) is stored on each phone. It does not sync between phones yet (see roadmap).

Roadmap

- Sync shop data to the server (a `shop_id` on every row) so several phones share one shop
- Staff accounts with roles (`owner`, `manager`, `cashier`). The role field already exists in `AppUser`
- PIN or biometric lock for shared counter phones

Fonts

Screen titles use **Pacifico**, bundled in `assets/fonts` under the SIL Open Font Licence (`OFL.txt`).

ShopDesk server (Node.js + Neon Postgres)

Authentication backend for the ShopDesk Flutter app: sign up with an emailed 6-digit code, sign in, forgot password, and sessions that stay signed in.

Runs on **Vercel**, **Docker / any VPS**, or your own PC. Data lives in **Neon Postgres**.

Features

- Sign up creates an unverified account and emails a 6-digit code
- Sign in is refused until the email is verified
- Forgot password: code first, then a one-time token unlocks the new-password step
- Short-lived access tokens (15 min) and rotating refresh tokens (30 days)
- Only one owner can ever register (switchable with `OWNER_ONLY`)
- Rate limits per email and per IP, lockout after repeated wrong passwords

Tech stack

Node 22.9+ (ESM) · Express 4 · `pg` (Neon Postgres) · `jsonwebtoken` · `nodemailer` · `helmet` · `express-rate-limit`. Tests use Node's built-in test runner against a real Postgres.

Quick start

```
cd shopdesk_server
npm install
cp .env.example .env     # fill in DATABASE_URL, JWT_SECRET, CODE_SECRET (see Settings)
npm start                # http://localhost:3000
```

Open `http://localhost:3000/health`. You should see `{"ok":true}` (it also checks the database). Tables are created automatically on first start.

1. Create the database (Neon)

1. Create a project at neon.com.
2. Press **Connect**, turn on **Connection pooling**, copy the string. It looks like `postgresql://user:pass@ep-xxxx-pooler.region.aws.neon.tech/neondb?sslmode=require`.
3. That string is your `DATABASE_URL`. It contains the password: keep it secret.

2. Settings (`.env`)

Name	Meaning
`DATABASE_URL`	Neon **pooled** connection string. Required.
`JWT_SECRET` , `CODE_SECRET`	Two different random strings, 32+ characters: `openssl rand -hex 48` . Required in production.
`SMTP_HOST` `SMTP_PORT` `SMTP_USER` `SMTP_PASS` `MAIL_FROM`	Your email sender (Resend, Postmark, SES, Mailgun, Gmail app password). Required in production.
`OWNER_ONLY`	`true` (default): one owner only. `false` : anyone can register.
`TRUST_PROXY`	`1` behind Render / Railway / Fly / Caddy / nginx. Set automatically on Vercel.
`PORT`	Default `3000` .

With `NODE_ENV=production` the server refuses to start unless the database, both secrets and SMTP are set, and the error says which one is missing. With no `SMTP_HOST` in development, codes are printed in the terminal.

API

All bodies are JSON. Errors look like `{"error":{"code":"invalid_code","message":"..."}}` .

Call	Body	Answer
`POST /auth/signup`	`email, password`	`{ok:true}` and a code is emailed
`POST /auth/verify-signup`	`email, code`	session
`POST /auth/resend-signup`	`email`	`{ok:true}`
`POST /auth/signin`	`email, password`	session, or `email_not_verified`
`POST /auth/forgot`	`email`	`{ok:true}` (same whether or not the account exists)
`POST /auth/verify-reset`	`email, code`	`{resetToken}`
`POST /auth/reset-password`	`resetToken, newPassword`	session (all other phones are signed out)
`POST /auth/refresh`	`refreshToken`	new session (token rotates)
`POST /auth/signout`	`refreshToken`	204
`GET /auth/me`	`Authorization: Bearer <accessToken>`	`{user}`
`GET /health`		`{ok:true}`

A session is `{user, accessToken, refreshToken, expiresIn}`. Error codes: `invalid_credentials`, `email_not_verified`, `invalid_code`, `code_expired`, `too_many_attempts`, `rate_limited`, `weak_password`, `sign_up_closed`, `invalid_request`, `unauthorized`.

Protect your own future routes with `app.locals.requireAuth`.

Security

- **Passwords:** hashed with scrypt, salted per password
- **Codes:** 6 digits, 10-minute life, single use, 5 wrong tries then dead. Stored only as an HMAC with a server secret. Tries are counted inside the database, so parallel guesses cannot get extra tries
- **No account leaks:** sign-up and forgot-password answer the same whether or not the email exists. Wrong email and wrong password give the same answer and take the same time
- **Email limits:** a new code at most every 30 seconds and 5 per hour, per address, counted for unknown addresses too
- **Reset:** the new-password step needs a one-time token that only the correct emailed code can produce
- **Lockout:** 8 wrong passwords lock that email for 15 minutes. Per-IP limits on top
- **Sessions:** refresh tokens are stored hashed and rotate on use. Replaying an old one signs the account out everywhere. Changing the password signs out every other device
- **Owner race:** two people verifying at the same moment cannot both become owner
- **Hardening:** `helmet` headers, 10 KB body limit, strict input validation (the server never trusts the app)

Deployment

Full steps in `DEPLOY.md`.

- **Vercel:** entry file is `src/app.js`. Add the environment variables in the Vercel dashboard and redeploy. Emails are sent with `waitUntil` so they still go out after the response.
- **Docker / VPS:** `docker compose up -d --build` starts the server plus Caddy, which gets and renews HTTPS for you.
- **Render / Railway / Fly.io:** use the `Dockerfile`. No disk needed, data is in Neon.

Project structure

```
src/
  app.js          Vercel entry (exports the app, builds it on first request)
  local.js        Normal entry: PC, Docker, VPS, Render, Railway
  create-app.js   The routes and rules, shared by both entries
  db.js           Neon connection pool, tables, atomic helpers
  security.js     scrypt, code hashing, input rules
  mailer.js       Email sending (prints codes in development)
  config.js       Settings and production checks
test/             26 tests against a real Postgres
Dockerfile · docker-compose.yml · Caddyfile · DEPLOY.md
```

Tests

```
npm test
```

Runs 26 tests against a **real Postgres**. With nothing set it starts a temporary one by itself (it cannot run as the root user). Or set `TEST_DATABASE_URL` to a **throw-away** database. The tests empty the tables, so never point it at your real Neon database.

Limits

- On Vercel the per-IP rate limit is kept per function copy, so it is weaker. The per-email limits live in the database and are fully enforced.
- Neon free projects pause when idle: the first request after a quiet period can take 1 to 3 seconds.
- If a phone loses the answer to a token refresh at exactly the wrong moment, the next refresh looks like a replay and the owner signs in again. Safe, slightly annoying.
- Shop data (products and sales) is not on the server yet. This server only handles accounts.

Deploying ShopDesk server

Target: one small Linux VPS (1 GB RAM is plenty), Docker, automatic HTTPS, and a Neon Postgres database. About 30 minutes. Any provider works (DigitalOcean, Hetzner, Linode, AWS Lightsail, Vultr). Ubuntu 24.04.

1. Before you start

1. **A domain** you control, for example `yourshop.com`. The API will live at `api.yourshop.com`.
2. **A transactional email service** with your domain verified: Resend, Postmark, Amazon SES or Mailgun. Add the SPF and DKIM records it gives you. Without them the codes land in spam. Copy its SMTP host, username and password.
3. **A VPS.** Note its public IP.
4. **A Neon database.** Follow step 1 in `README.md` and keep the pooled connection string (`DATABASE_URL`).
5. In your domain's DNS add an **A record**: `api` → the VPS IP. Wait until it resolves.

2. Prepare the server

```
ssh root@YOUR_IP
apt update && apt -y upgrade
curl -fsSL https://get.docker.com | sh
ufw allow OpenSSH && ufw allow 80 && ufw allow 443 && ufw --force enable
```

(Better: create a non-root user, add it to the `docker` group, and disable password SSH login.)

3. Put the code on the server

From your computer, copy the `shopdesk_server` folder (not `node_modules`):

```
scp -r shopdesk_server root@YOUR_IP:/opt/shopdesk
```

4. Configure

```
cd /opt/shopdesk
cp .env.production.example .env
nano .env        # fill in every empty value
```

Paste the Neon string into `DATABASE_URL`. Make the two secrets with `openssl rand -hex 48` (run it twice, different values). Use the SMTP details from your email service and set `API_DOMAIN`.

5. Start

```
docker compose up -d --build
docker compose logs -f api     # expect: "ShopDesk auth server on :3000 (production,
ownerOnly=true, db=postgres)"
curl https://api.yourshop.com/health     # {"ok":true}
```

Caddy fetches the HTTPS certificate on its own the first time (needs ports 80 and 443 open and DNS ready). The server refuses to start in production if a secret or SMTP is missing, and says which.

6. Build the app against it

```
flutter build apk --release --dart-define=API_URL=https://api.yourshop.com
flutter build appbundle --release --dart-define=API_URL=https://api.yourshop.com   # Play
Store
flutter build ios --release --dart-define=API_URL=https://api.yourshop.com
```

The app no longer adds sample data, so every shop starts empty. A phone that already ran an older test build still holds the old sample products: uninstall the app (or clear its storage) once before release.

7. Create the owner

Open the released app, tap Create owner account, enter the emailed code. The first verified account becomes the owner and sign-up closes for everyone else.

Running it

Task	Command
Logs	`docker compose logs -f api`
Update after code changes	copy the new files, then `docker compose up -d --build`
Restart	`docker compose restart api`

Database and backups. All data is in Neon, not on this server, so rebuilding the server loses nothing. In the Neon console check your plan's restore window (point-in-time restore) and, if you want, export a `pg_dump` now and then: `pg_dump "$DATABASE_URL" > backup.sql`.

Lost the owner password and email? Reset needs email access. If the email address itself is lost, delete the user row in the Neon SQL Editor (`DELETE FROM users;`) and register again.

Other hosts

- **Render / Railway / Fly.io:** use the `Dockerfile`. Set the variables from `.env.production.example` (skip `API_DOMAIN`) and `TRUST_PROXY=1`. No disk is needed because the data is in Neon. Health check path: `/health`. Several instances are fine.

Deploying on Vercel

The server works on Vercel (entry file `src/app.js`, which Vercel finds on its own). Your own Docker/VPS and local runs use `src/local.js` instead.

1. Put the `shopdesk_server` folder in your Git repo and import it in Vercel. If the server folder is not the repo root, set **Project Settings → Root Directory** to that folder.
2. **Settings → Environment Variables**, add (for Production): `DATABASE_URL` (Neon **pooled** string), `JWT_SECRET`, `CODE_SECRET`, `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `MAIL_FROM`, `OWNER_ONLY=true`.
3. **Redeploy** (changing variables does nothing until you redeploy).
4. Open `https://YOUR-PROJECT.vercel.app/health`. You should see `{"ok":true}`. If you see an error, open **Logs** in Vercel: a missing variable is named in the message.

Good to know on Vercel: - The first request after a quiet period is slower (the function and Neon both wake up). - Emails are sent after the answer goes back, using Vercel's `waitUntil`, so they still arrive. - The per-IP limit is kept per function copy, so it is weaker here. The per-email limits (code sends, wrong passwords, wrong code tries) live in the database and are fully enforced. - Outbound SMTP on port 587 or 465 works. Port 25 does not.

Production checklist

- [ ] DNS A record resolves, `https://api.../health` works with a valid certificate
- [ ] `JWT_SECRET` and `CODE_SECRET` are different 96-character random values
- [ ] Test email arrives in the **inbox**, not spam (send a code to Gmail and Outlook addresses)
- [ ] App built with the `https://` `API_URL`, installed fresh (no old test data on the phone)
- [ ] Owner account created; confirm a second sign-up attempt shows "turned off"
- [ ] Neon restore window checked (or a `pg_dump` saved)
- [ ] `DATABASE_URL` is the **pooled** string and is only in `.env` on the server
- [ ] Only ports 22, 80, 443 open